

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:50

Mode: In Class

Hours : 60 mts

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Challa Priyadarshini (192311118) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Annexure A

ANALYTICAL PROBLEM SOLVING

- 1 Trace the input expression $y = (p + q) * r - s / t$ through all phases of a compiler by identifying the tokens generated during lexical analysis, constructing the syntax tree during parsing, generating intermediate code, applying possible optimizations, and interpreting the relevance of the final target code produced.
- 2 Analyze the tokenization process of the expression `float total = value--++ + ++count-- * 10.5;` by identifying ambiguities in operator recognition, explaining how a LEX-based lexical analyzer processes such patterns, modifying the rules to resolve conflicts, and justifying the correctness of the updated lexical specification.
- 3 Determine the total number of tokens present in a given C program that includes nested loops, multiple variable declarations, arithmetic expressions, and inline comments, while carefully considering how the lexical analyzer ignores comments and whitespace. Analyze each line of the program to identify keywords, identifiers, operators, constants, and punctuation symbols as individual tokens, and compute the final token count based on proper lexical classification.

```
#include <stdio.h>
```

```
int main() {  
    int i, j, sum = 0;    /* initialize variables */  
  
    for(i = 0; i < 3; i++) {  
        for(j = 0; j < 2; j++) {  
            sum = sum + i * j;  
        }  
    }  
  
    printf("%d\n", sum);  
    return 0;  
}
```

- 4 Construct a non deterministic finite automata with ϵ -transitions to represent the language denoted by the regular expression $10+(0+11)^*0^*1$
- 5 Construct the language for the following Regular Expressions:
 - a) $(\epsilon+a)^*$
 - b) $(a+b)^*aa$
 - c) a^*ba^*
 - d) $(ab+a)^*$
 - e) $(ab)^*+(ba)^*$

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

COI - AT1 - Analytical problem Solving

compiler Design csA1415

given input expression

$$y = (p+q) * r - s / t$$

1. Lexical analysis.

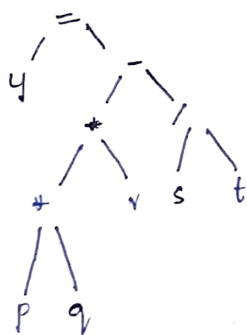
$$y = (p+q) * r - s / t$$

$\langle id, 1 \rangle \langle + \rangle \langle id, 2 \rangle \langle + \rangle \langle id, 3 \rangle \langle * \rangle$
 $\langle id, 4 \rangle \langle - \rangle \langle id, 5 \rangle \langle / \rangle \langle id, 6 \rangle$

Lexems	tokens
y	identifier
=	operator
(	identifier symbol
p	identifier
+	operator
q	identifier
)	symbol
*	operator
r	identifier
-	operator
s	identifier
/	operator
t	identifier

Total tokens = 13.

2. Syntax Analysis



3. Semantic Analysis

Here all same as syntax analysis.

4. Intermediate code.

$$t_1 = p + q$$

$$t_2 = t_1 * r$$

$$t_3 = s / t$$

$$t_4 = t_2 - t_3$$

$$y = t_4$$

5. code optimization

Already the code is optimized.

6. code generator

```
MOV R1, s
MOV R2, t
DIV R1, R2
MOV R3, v
SUB R3, R1
MOV R4, p
MOV R5, q
ADD R4, R5
MUL R4, R3
MOV y, R4
```

Tokenization process of expression.

float total = value -- ++ ++ count -- * 10.5 ;

Tokens

Lexeme	Token
float	keyword
total	identifier
=	operator
value	identifier
--	Decrement
++	Increment
+	operator
++	Increment
count	Identifier
--	Decrement
*	Operator
10.5	constant
;	Symbol

Ambiguity

problem:

value -- ++

interpreted as

value -- ++

But this is not valid in C because the result of value -- is not an int value.

Similarly, ++count-- is invalid.

LEX Rule processing

"--" { return DEC; }

"++" { return INC; }

"+" { return PLUS; }

"-" { return MINUS; }

Lex recognizes

value

-- independently.

It only recognizes tokens. It doesn't check syntax, parser reports the error.

Justification

- Longest matching rule removes ambiguity, lexical analyzer identifies operators correctly, parser later detects invalid operator combinations.

Total No. of Tokens.

```
#include <stdio.h>
```

```
int main() {
```

```
    int i, j, sum = 0;
```

```
    for (i = 0; i < 3; i++) {
```

```
        for (j = 0; j < 2; j++) {
```

```
            sum = sum + i * j;
```

```
        }
```

```
    }
```

```
    printf("%d\n", sum);
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int i, j, sum = 0;
```

```
    for (i = 0; i < 3; i++) {
```

```
        for (j = 0; j < 2; j++) {
```

1

5 int, main, (, {, }

9 int, i, j, sum, =, 0, ,, ,, ,, ,

14 for, (, i, =, 0, ,, i, <, 3, ,, i, ++,), {,

14

sum = sum + i * j ; } } } 10 sum = , sum , + , i , * , j , ; , } , }

printf("%d\n", sum); printf(, "%d\n" , , sum , ,

return 0; } return , 0 , ; , }

Total no. of tokens = 65.

4. given finite automata with ϵ -transitions:

The regular expression $10 + (0 + 11)^* 0^*$

$\Rightarrow 10 + (0 + 11)^* 0^*$

Equivalent: $(10) + ((0 + 11)^* 0^*)$

$+$ = union

$*$: Kleene star

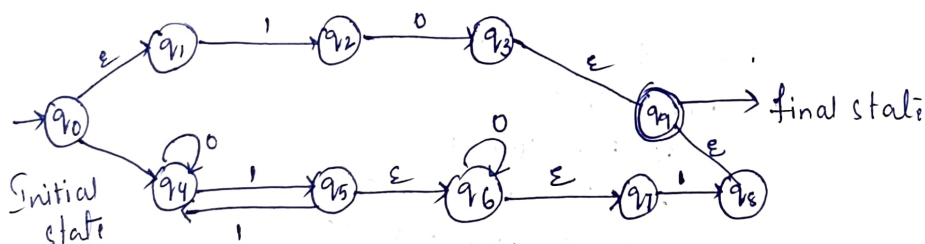
ϵ : epsilon transition

The regular expression has 2 parts.

1. 10

2. $(0 + 11)^* 0^*$

ϵ -NFA:-



5.

given Regular Expressions:

a) $(\epsilon + a)^*$

ϵ means empty string

$L = \{\epsilon, a, aa, aaa, aaaa, \dots\}$

b) $(a + b)^* aa$

$(a + b)^*$ any combinations of a & b (including empty string)

Every string must end with "aa".

d: $\{ waa | w \in \{a,b\}^* \}$

aa
baa
abaa
bbaa

c) (a^*ba^*) .

strings containing exactly one b with any no. of a's before and after.

ex:

b
ab
ba
aab
baa
aaabaa

d) $(ab+a)^*$

all strings formed using a or ab repeatedly.

Examples:

ϵ
a
ab
aa
aab
aba
abab.

e) $(ab)^* + (ba)^*$

strings consisting of repeated ab (or) repeated ba.

ex:-

ϵ
ab
abab
ababab
ba
baba
bababa.